SAVONIA

THIS IS THE WAY TO CHANGE THE WORLD.

PROJECT REPORT – BACHELOR'S DEGREE
TECHNOLOGY, COMMUNICATION AND TRANSPORT

REMOTE CONTROLLED ROBOT WITH OBSTACLE DETECTION

Group 4

AUTHOR/S Eemeli Vartio

Niklas Etling

Santeri Karjalainen

Sandesh Neupane

Yetunde Abdulazeez- Ado

Zohaib Salman

SAVONIA UNIVERSITY OF APPLIED SCIENCES

THESIS
Abstract

Field of Study Technology, Communication and Transport	
Degree Programme Degree Programme in Incormation Technology, Internet of Things	
Author Santeri Karjalainen, Niklas Etling, Sandesh Neupane, Yetunde Abdulazeez- Ado, Zohaib Salman, Eemeli Vartio	
Title of Thesis	
Date 08.12.2025	Pages/Appendices
Client Organization /Partners	

The objective of this project is to develop and produce an embedded IoT system that can be used on the iRobot create 3 platform to turn it into a remote controllable, object avoiding robot using sensors and microcontrollers in a local network controlled wirelessly on a web-interface. The team utilized STM32 microcontroller, Time-of-Flight sensors, Raspberry Pi's and various programming languages in the development.

The group worked as a team utilizing available communication platforms, sharing information effectively and helping each other through every problem.

This project could be used as a universal modifiable platform to develop cost and power efficient robotics for customers. Conclusion from the project is that the methods and technologies are viable for the intended purpose and future development is possible. The group were able to create a working prototype.

This project was made as part of IoT project 1 course for Savonia University of Applied Sciences.

Keywords
Embedded IoT System, Remote Control, iRobot Platform, Sensor Integration, Collision Avoidance System

CONTENTS

1 INTRODUCTION 5
2 SYSTEM DESIGN AND ARCHITECTURE 6
2.1 System overview 6
2.2 Description of parts technologies 7
2.2.1 Software for parts 7
2.3 Web-interface Programming 8
3 IMPLEMENTATION 9
3.1 Development process 9
3.1.1 Hardware 9
3.1.2 Web Interface Development 11
3.1.3 Backend Server & Communication Setup 14
3.2 Movement control application 14
3.3 Problem solving 14
3.3.1 Web Interface Challenges and Solutions 14
3.3.2 Microcontroller and Sensor challenges 15
3.3.3 Wireless communication 15
4 TESTING AND RESULTS 16
4.1 Testing 16
4.1.1 Circuit and Communication Smoke Check 16
4.1.2 Testing the web application 16
4.2 Results 17
4.2.1 Smoke Test Results 17
4.2.2 Web application 17
5 DISCUSSION 19
5.1 Project objectives 19
5.2 Contribution of project members 19
5.3 Future development 19
REFERENCES 20
APPENDIX 1: CREATIVE COMMONS LICENCES – IMAGE COPYRIGHT 21

LIST OF FIGURES

Figure 1. System architecture 6
Figure 2. ToF sensor 8 x 8 distance array 8
Figure 3. Sensor array wiring diagram 10
Figure 4. Function for handling the virtual joystick 12
Figure 5. Function for keyboard control 13
Figure 6. VL53L8CX sensor array connected via I2C multiplexer 16
Figure 7. Six sensor system console log 17
Figure 8. Application 18
Figure 9. Gantt chart 19

1 INTRODUCTION

This report presents the development process of collision avoidance system build on top of iRobot educational platform. The idea and motivation were to develop a platform for custom building autonomous/semi-autonomous robots that can be tailored to customer needs.

Project aim during this IoT project 1 course is to use the iRobot platform to build a device that can be remotely controlled but also has autonomous safety overrides, such as recognizing when the robot is near a ledge about to fall or on a collision course. These features are implemented with time-of-flight sensors referred to as ToFs in this report. We plan to make a web application for manually controlling the robot with premade library of movement control commands from iRobot. Local network is used in facilitating wireless communications between the web application and physical components.

Movement of the robot is manual, but automatic safety overrides are implemented. For this time-of-flight sensors are used to have a sense of the robots' surroundings. If the movement seems unsafe or going into unsafe conditions the STM32 with the time-of-flight sensors will send an override message to stop the robot movement.

Chapter 2 goes over the whole system describing what parts and technologies were chosen and why. Chapter 3 focuses on how these parts and technologies were implemented to fulfil the goal of the project. Chapter 4 is showcasing the results and chapter 5 discusses the project.

2 SYSTEM DESIGN AND ARCHITECTURE

2.1 System overview

For this project the key is control of the physical system. This is established with robot operating systems 2 aka ROS 2. ROS 2 “is a set of software libraries and tools for building robot applications.”. (Open Robotics, 2025).

The robot system itself is iRobot create 3 educational robot platform. From this we get prebuilt platform that has everything required for robot to move. It is based on ROS 2 and allows students, teachers and developers to build up from there (iRobot Education 2022, 1).

Aside from developing an interactive user interface for the backend which provides ability to remote control the robot platform, the main goal of the project was to create a collision avoidance system. During this project this is implemented with STM VL53L8CX time-of-flight sensors. These sensors are attached to a NUCLEO-144 STM32H723ZG EVAL board with Adafruit TCA9548A 1-to-8 I2C Multiplexer.

The embedded system communicates with the remote-control application inside a local network created by utilizing OpenWRT with Raspberry Pi 5.

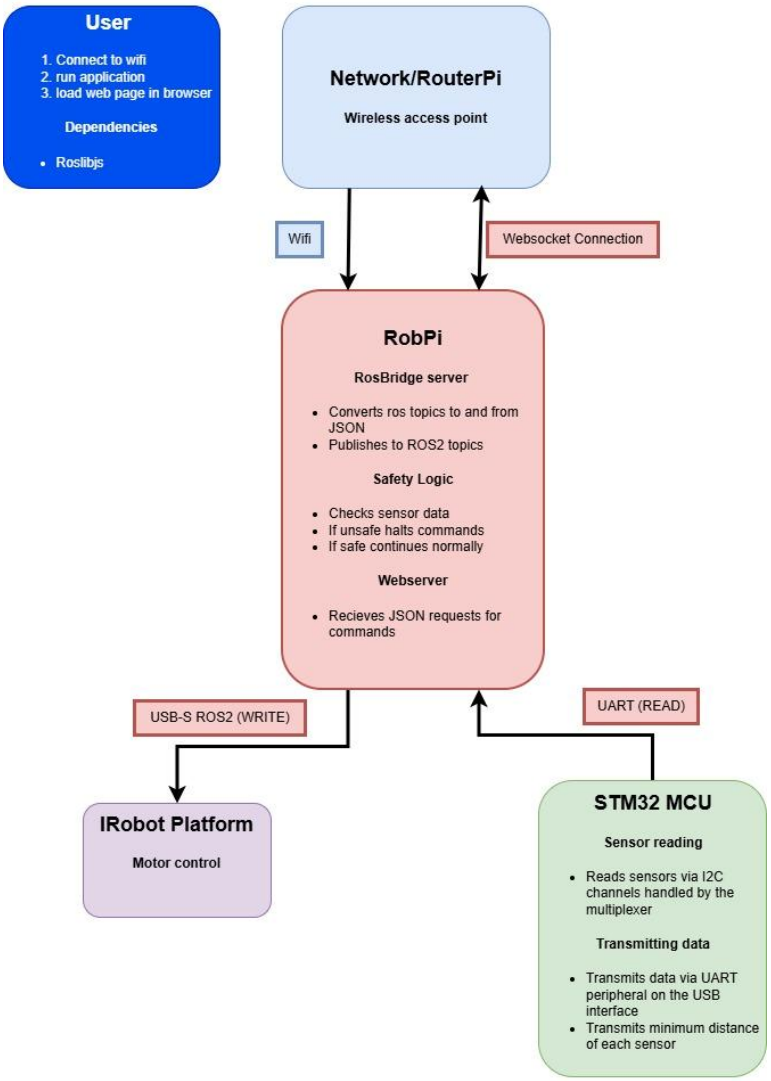


Figure 1. System architecture

2.2 Description of parts technologies

STM32 Nucleo-144 board with H723ZG was chosen as the microcontroller unit for processing sensor data. The STM32 Nucleo-144 board provides an affordable and flexible way for users to try out new concepts and build prototypes by choosing from the various combinations of performance and power consumption features, provided by the STM32 microcontroller (STMicroelectronics, 2025).

The iRobot Create 3 was chosen as the platform for movement, as the team had previous knowledge of its operation. This allowed the team to focus on the main goal, which was to develop a system for collision avoidance. The platform also has a wooden body built on top of it which was utilized for mounting sensors at varying heights around the body.

To detect approaching collision, the team wanted to use time-of-flight (ToF) sensors. VL53L8CX ToF sensors provide the ability to detect objects and their distance from the sensor with multiple zones, which is different from traditional single-point ToF. In recent years, the usual approach for this task is to utilize lidar systems, many of which use ToF principles but involve rotating mechanisms to scan the environment, are more expensive and use more processing power. Using these sensors with our chosen STM32 MCU, it is possible to achieve collision avoidance for safety and in the future, simultaneous localization and mapping (SLAM) for autonomous navigation, with efficient power usage and lower cost. The team wanted to focus on the main task of sensor fusion and communication between devices without the need to design custom PCBs for the sensors. Solution was to use dedicated breakout boards, which integrate the sensor and provide an easy way to connect pins. SATEL VL53L8 breakout board was chosen for this project.

A Raspberry Pi 5 was utilized to create a local network using OpenWRT. This enables high-speed communication between the devices.

Inside the iRobot Create 3, a Raspberry Pi 4 was used as middleware for motor control. The Pi is used to communicate with the Create 3's internal computer which is running ROS2. Pi 4 also creates a ROS2 WebSocket server for communication between software components.

2.2.1 Software for parts

The embedded software running on the STM32 microcontroller is responsible for the low-level management of the sensor system. The firmware is developed in C using the STM32Cube Hardware Abstraction Layer (HAL) to ensure portability and robustness. To interface with the time-of-flight (ToF) sensors, the system uses the official VL53L8CX Ultra Lite Driver (ULD). A custom platform layer was implemented to bridge the ULD's generic API calls with the specific STM32 I2C hardware, particularly handling the 16-bit memory addressing required for reading and writing to sensor registers. (STMicroelectronics 2023; STMicroelectronics 2025.)

A core function of the software is managing the I2C multiplexing to communicate with six sensors all containing the same address. The firmware implements channel selection logic where a specific bitmask is written to the TCA9548A multiplexer to choose which channel will be used for I2C communication. During the initialization phase, the system first performs a hard reset by toggling the Low Power enable (LPn) pin.

Unlike traditional sensors with non-volatile memory, the VL53L8CX ToF sensor is a RAM-based device that requires firmware to be uploaded at every power cycle. The ULD includes a dedicated

header file. buffers.h, which contains the sensor's firmware as a hexadecimal array. During the boot sequence, the STM32 transmits the firmware blob to each sensor over the I2C lines. This process is required for sensor operation but introduces a delay during the startup phase, requiring approximately five seconds per each sensor. After initialization, each sensor is configured to operate at a ranging frequency of 15 Hz with an 8x8 zone resolution for optimal precision. (STMicroelectronics 2023.)

```
Detailed View (8x8 Zone Data - Sensor 1):
----------------------------------------------------------------
Zone |  0  |  1  |  2  |  3  |  4  |  5  |  6  |  7  |
----------------------------------------------------------------
Row 0|  495|  506| 1995| 2006| 1999| 2001| 2007| 1994|
Row 1|  502|  494| 2002| 1997| 2005| 2000| 1996| 2003|
Row 2|  499|  501| 1994| 2001| 1998| 2006| 2002| 1999|
Row 3|  507|  493| 2007| 1993| 2004| 2000| 1995| 2001|
Row 4|  496|  505| 2001| 2005| 1997| 2003| 1999| 1996|
Row 5|  500|  498| 1999| 2002| 2006| 1994| 2001| 2005|
Row 6|  503|  497| 2003| 1996| 2000| 2007| 1995| 1998|
Row 7|  494|  502| 2000| 2004| 1993| 1998| 2006| 2002|
----------------------------------------------------------------
Min: 493mm | Max: 2007mm | Avg: 1625mm | Valid: 64/64 | Status: OK
```

Figure 2. ToF sensor 8 x 8 distance array

The 8x8 resolution provides a dense matrix of 64 distance measurements per sensor. The main application loop operates as a data processor. It cycles through the multiplexer channels, polling the sensors for ready data. Once a measurement is completed, the firmware retrieves the ranging results and immediately processes the data. The code iterates through the 64 zones of the 8x8 array, filtering out any zones deemed invalid, and extracting the absolute minimum distance. This value is formatted into a comma- separated serial string and transmitted via UART to the Raspberry pi. (STMicroelectronics 2023.)

2.3 Web-interface Programming

We use several key technologies, each chosen for a specific benefit:

- **ROSLIB (Robot Web Tools, 2025.):** Essential for connecting the web browser to ROS 2 via rosbridge. It provides simple, robust integration and communication methods (topics, services, actions) directing in JavaScript.
- **React (Meta, 2025.):** Used for its efficiency in building interactive UIs, making it an excellent choice for creating the stateful and responsive virtual joystick control.
- **Python (Python Software Foundation, 2025.):** The core language for ROS 2 nodes, which maintains consistency across the entire system

3 IMPLEMENTATION

3.1 Development process

At the start of the project, after finalizing the idea a project scope was made of what to accomplish during the project development period. This started by looking at the movement of the robot, looking at variety of options and landed on working with the iRobot educational development kit. This kit already had a stand made of wood that sensors can be attached on. iRobot also has ready-made movement controls that were used for controlling the robot.

The team continue to pick components and establish communication channels between group members. WhatsApp was chosen for day-to-day communication and Discord for sharing resources. Documentation is written during and after every development session to keep up to date with progress. This way the team had something to refresh and rely on when writing the final report for the project.

Tasks were divided into three parts web-interface programming, device communication and setting up microcontrollers with the sensors. Individuals in the group were able to choose the tasks that were in their own interest and skill range. Focus still is that everyone contributes and understands the functionality of the project.

3.1.1 Hardware

The Raspberry Pi 4 inside the iRobot Create 3 functions as middleware between the higher-level control systems and the robot's motor controllers. This board runs ROS2 Humble and communicates with the Create 3's internal computer via USB-C connection. It has sufficient processing power to handle multiple communication protocols simultaneously. Ubuntu 22.04 Server OS running on the Pi 4 provides environment capable of running ROS2, which also hosts a WebSocket server that enables remote control application to send commands to the robot, translating high-level navigation commands into ROS2 messages that the Create 3's internal systems can execute.

The Raspberry Pi 5 is the network backbone of the system, running OpenWRT to create a local network. The system needed a reliable low-latency local network for communication between all system components without dependence on external internet connectivity. The Pi 5's networking capabilities and processing power make it a decent option for handling multiple simultaneous connections between the devices. The local network was especially useful for development of the system since it provides ability to easily remote access the devices without interference. This was a solution proposed by another student in our class Min Set Ko. The problem was that the schools' network did not allow communication between devices and when we came across this problem Min Set helped us with the idea and the setting up the OpenWRT with the Raspberry Pi 5.

STM32 Nucleo-144 H723ZG serves as the sensor processing unit in the system. High-performance ARM Cortex-M7 core running at 550MHz provides the computational power for real-time sensor data processing, and it maintains low power consumption compared to larger embedded computers. The H723ZG is responsible for interfacing with the VL53L8CX ToF sensors through I2C communication, processing incoming distance data. Its real-time processing capability ensures minimal latency in obstacle detection, which is critical for safe navigation through varying environments.

The system utilizes six VL53L8CX ToF sensors arranged around the robot's perimeter to provide near 360-degree obstacle detection coverage. In real world environments, full 360-degree precision is not necessary for the task of collision avoidance, since the obstacles typically have a wider profile than the 'dead zones' between sensor fields of view, and the robot is constantly moving. For future development, 8 sensors will be used to provide complete 360-degree coverage, which will enable the ability to map the environment with high precision. Each VL53L8CX sensor provides an 8x8 zone array, allowing it to detect multiple objects simultaneously within its field of view and measure distances up to 4 meters with millimetre resolution in optimal conditions (dark target, no ambient light). In typical conditions, reliable ranging of 1-3 meters is more than sufficient for the purposes of this project as the system operates in controlled indoor environments.

A challenge in using multiple VL53L8CX sensors is that they share the same default I2C address (0x29). To solve this, the system uses a TCA9548A I2C multiplexer, which provides eight independent I2C channels. The multiplexer connects to the STM32's I2C bus and allows the microcontroller to communicate with individual sensors by switching between channels.

The wiring architecture follows a hub-and-spoke topology with the I2C multiplexer at the centre. The STM32 Nucleo board's I2C pins (SCL and SDA) connect to the multiplexer's I2C interface. Each VL53L8CX sensor connects to one of the eight available channels on the multiplexer, with each channel providing individual SCL and SDA lines. All sensors share common ground and power connections. The SATEL VL53L8 breakout board has internal pull-up resistors to ensure signal integrity. The STM32 Nucleo board controls which sensor it communicates with by sending channel selection commands to the multiplexer, creating isolated communication paths to each sensor despite their identical addresses.

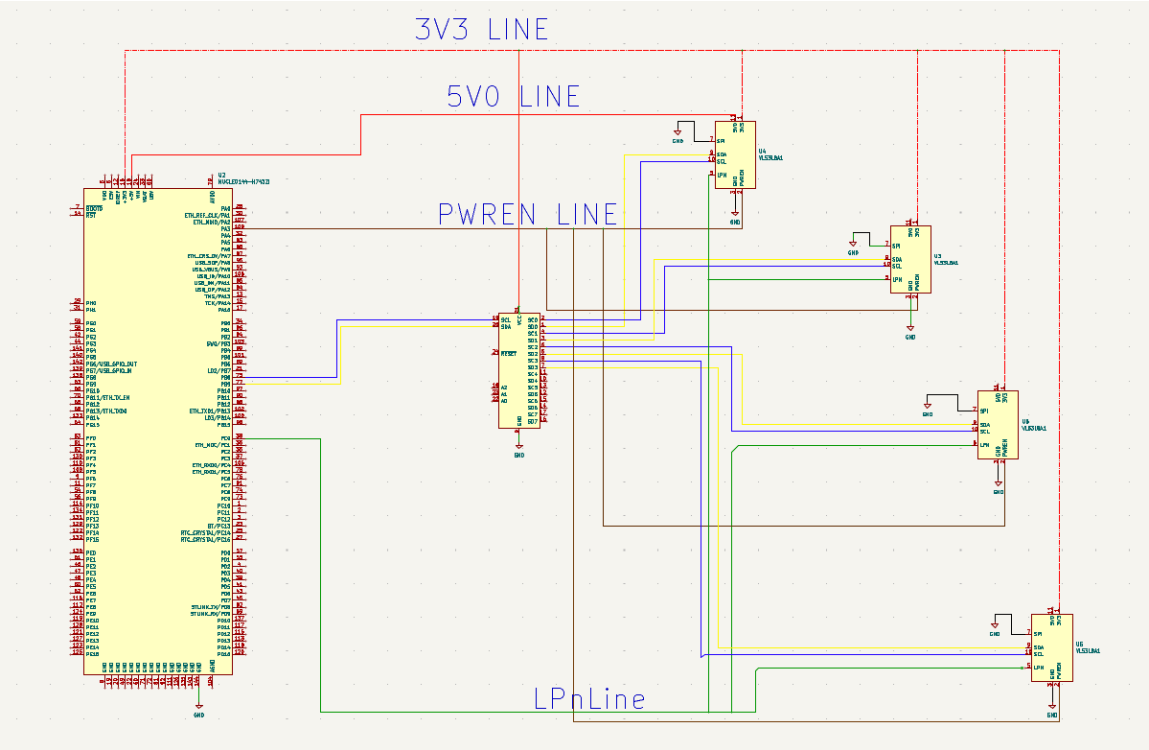


Figure 3. Sensor array wiring diagram

The STM32 Nucleo board can be powered via USB or through its onboard voltage regulators. The VL53L8CX sensors operate at 3.3V and draw current of approximately 20mA each. The I2C multiplexer also operates at 3.3V and consumes minimal power in its switching operations. During testing of an external powering system which involves 2x 18650 rechargeable batteries, a short circuit occurred in the battery casing. This resulted in an overcurrent condition that exceeded the protection limits of the STM32 board's voltage regulators. The resulting current surge damaged the Nucleo board. With the project deadline one week away, the team did not have enough time to order a replacement board. One team member had an STM32H747I-DISCO development kit available, and the system was successfully migrated to this platform. The H747I differs as it has a dual-core architecture, but the main processor is the same (Cortex-M7). As sensors and the multiplexer remained functional and the wiring for the system was already done, this allowed the development to continue.

3.1.2 Web Interface Development

The IrobotController.tsx (ZohSal, 2025.) file serves as the core of the robot's web interface, implemented as a React TypeScript component. This component facilitates all communication and control logic for the robot, leveraging modern web technologies to provide a smooth, cross-platform user experience.

The most critical function of this software is establishing and managing the connection to the ROS 2 running on the Raspberry Pi. The web interface imports the roslibjs library (Robot Web Tools, 2025.), which is the standard JavaScript tool for interacting with ROS over the web. This library enables the use of the WebSocket protocol to communicate with the ROS Bridge Server running on the robot

Primary useEffect hook manages the connection state, it creates a ROSLIB.Ros object when the connected state is true, connecting to the Pi's ROS Bridge server at 192.168.1.219:9090. It registers event listeners to dynamically update the user-facing connection status.

Velocity Publishing Topic: A ROSLIB.Topic object is instantiated for the standard movement command topic, /cmd_vel, using the geometry_msgs.msg/Twist message type. This topic is the conduit for sending linear (forward/backward) and angular(turning) velocity commands to the robot base.

The application supports both virtual joystick and keyboard input for control, ensuring redundancy and compatibility across various devices.

Continuous Publishing (10Hz): the core of the smooth control is the startMoving function. Instead of sending a single command, it uses setInterval to continuously publish the current velocity commands to the /cmd_vel topic every 100 milliseconds (10Hz). This high refresh rate ensures the robot moves smoothly and responds quickly to control changes, meeting the less than 100 Ms response time requirement (as publishing occurs at 10 Hz). The stopMoving function clears this interval and publishes a final zero-velocity command to ensure the robot stops instantly.

Virtual joystick is handled by handleJoystickMove. It calculates the X and Y offset of the joystick knob relative to the center. The linear velocity is mapped to the Y-axis movement. This allows for simultaneous movement and rotation.

```
// Handle joystick movement
const handleJoystickMove = (e: MouseEvent<HTMLDivElement> | TouchEvent<HTMLDivElement>) => {
  if (!isDragging || !joystickRef.current) return;

  const rect = joystickRef.current.getBoundingClientRect();
  const centerX = rect.width / 2;
  const centerY = rect.height / 2;

  const clientX = 'touches' in e ? e.touches[0].clientX : e.clientX;
  const clientY = 'touches' in e ? e.touches[0].clientY : e.clientY;

  let x = clientX - rect.left - centerX;
  let y = clientY - rect.top - centerY;

  const distance = Math.sqrt(x * x + y * y);
  const maxDistance = centerX * 0.8;

  if (distance > maxDistance) {
    x = (x / distance) * maxDistance;
    y = (y / distance) * maxDistance;
  }

  setJoystickPos({ x, y });

  // Joystick up (negative y) = reverse (positive linear vel)
  // Joystick right (positive x) = turn left (positive angular vel)
  const linearVel = (y / maxDistance) * speed;
  const angularVel = (x / maxDistance) * turnRate;
```

Figure 4. Function for handling the virtual joystick

A second useEffect hook manages global keyboard events. The moveBindings object map keys (‘l’, ’,’, ’.’, ’m’) to specific directional vectors (linear X, linear Y, angular X, angular Z). it tracks pressed keys using pressedKeysRef to ensure continuous movement while a key is held down and to prevent multiple keypresses from interfering. It also includes key bindings (‘q’, ‘z’, ‘w’, ‘x’, ‘e’, ‘c’) for dynamically adjusting the robot’s linear and angular speed in real-time.

```
const handleKeyDown = (e: KeyboardEvent) => {
  // Movement commands
  if (key in moveBindings && !pressedKeysRef.current.has(key)) {
    pressedKeysRef.current.add(key);
    setActiveKey(key);
    const [x, y, z, th] = moveBindings[key];
    const linearVel = x * speed;
    const angularVel = th * turnRate;
    startMoving(linearVel, angularVel);
  }
};

const handleKeyUp = (e: KeyboardEvent) => {
  const key = e.key;
  if (key in moveBindings && pressedKeysRef.current.has(key)) {
    pressedKeysRef.current.delete(key);
    setActiveKey('');
    stopMoving()
  }
};

window.addEventListener('keydown', handleKeyDown);
window.addEventListener('keyup', handleKeyUp);

return () => {
  window.removeEventListener('keydown', handleKeyDown);
  window.removeEventListener('keyup', handleKeyUp);
};
```

Figure 5. Function for keyboard control

The user interface provides an interactive control system accessible across computers, tablets, and smartphones. Central to the design is a virtual joystick (CircuitAnalysis, 2021.) that allows users to drag with a mouse or touch to control both forward/backward movement and turning. For immediate control, the joystick automatically returns to the center when released.

The interface also includes a dedicated status display which shows the current connection status, the robot's ongoing action, and any error messages, providing quick diagnostic feedback.

The system is structured using HTML5, employs React (Meta, 2025.) for building interactive components like the joystick, utilizes Tailwing CSS (Tailwind Labs, 2025.) for a clean modern aesthetic, and runs on JavaScript for underlying logic and communication with ROS 2 via ROSLIB (Robot Web Tool, 2025.).

3.1.3 Backend Server & Communication Setup

We develop a Flask server (Pallets, 2025.) that runs on the Raspberry Pi. Its primary role is to act as the lightweight host for the web application and to ensure the necessary background processes, including the rosbridge_server, are running. This entire system utilizes ROS 2 as its underlying robotics framework (Macenski, T., Foote, B. Gerkey, C. Lalancette, W. Woodall, 2022.).

Crucially, all dynamic robot communication is handled by ROSLIB (Robot Web Tools, 2025.) connecting directly to the rosbridge_server. The flask application serves the static web interface files (HTML/React/JS) to the user's browser but does not process the real-time velocity or action commands through its own API endpoints.

The implementation relies on Python 3 (Python Software Foundation, 2025.), the Flask web framework (Pallets, 2025.), and integrates directly with ROS 2 Humble (Open Robotics, 2025.) through the intermediary of the rosbridge_server.

3.2 Movement control application

We created a web-based control system for the iRobot Create 3 robot that can be accessed from any device (computer, phone, or tablet) through a web browser. This replaces the traditional keyboard control method with a more user-friendly interface. The web-based controller interface connects to the robot through the Raspberry Pi.

Moving forward data flow example:

1. User drags joystick forward in web browser
2. JavaScript calculates velocity values (linear: 0.5, angular: 0.0)
3. Browser sends POST request to ROSLIB which has the robots IP address stored
4. ROSLIB receives the request
5. Server publishes velocity to /cmd_vel topic in ROS 2
6. ROS 2 sends the command to iRobot Create 3
7. Robot moves forward

3.3 Problem solving

In the development process, the team encountered multiple new technologies and complex hardware to software integrations. This led to various challenges and required us to work as a team to solve these challenges in the project time frame. These challenges spanned from application-level logic, hardware initialization to wireless communication.

3.3.1 Web Interface Challenges and Solutions

The initial primary obstacle was connecting the client-side web interface to the ROS 2 robotics framework. Web browsers, by design, cannot directly communicate with ROS. Our initial approach attempted to use a dedicated Flask server running on the Raspberry Pi to act as a bridge. This server was intended to receive standard HTTP requests from the browser and convert them into

ROS 2 commands. However, this method introduced a subsequent and more critical issue: synchronization and latency. The Flask server proved inefficient at maintaining the 10Hz command rate expected by ROS 2 for smooth control, often resulting in a slower transmission rate that caused an exponential delay and jerky robot motions.

To resolve the synchronization and command rate problems, we completely shifted the real-time communication strategy. The solution was the integration of the ROSLIB JavaScript library which connects directly to the rosbridge_server running alongside ROS 2 (Robot Web Tools, 2025.). ROSLIB follows the native ROS 2 communication patterns, allowing the web interface to publish velocity commands directly to the /cmd_vel topic at the required high frequency, thus eliminating the synchronisation mismatch caused by the Flask server's HTTP request-response cycle. This allowed us to achieve significantly smoother robot movement and near-minimal control latency, ensuring a responsive user experience. The Flask server (Pallets, 2025.) was consequently relegated to its core role: serving the static web files and ensuring background processes, like the rosbridge_server, were running.

Two final challenges related to network accessibility and browser security had to be addressed to ensure the system was universally accessible. Initially, the team struggled with making the web interface accessible from devices other than the host. This was resolved by configuring the Raspberry Pi to allow network hosting using its static IP address and the configured rosbridge_server port (9090) for client connections (Macenski, T., Foote, B. Gerkey, C. Lalancette, W. Woodall, 2022.). Furthermore, security protocols presented an issue: web browsers blocked requests from the front-end interface to the rosbridge_server due to Cross-Origin Resource Sharing (CORS) restrictions. This was fixed by implementing Flask-CORS (Flask-CORS Documentation, 2025.) on the server, which explicitly allows cross-origin requests, enabling the browser-based ROSLIB connection to successfully establish and maintain communication with ROS 2 core.

3.3.2 Microcontroller and Sensor challenges

The primary challenge in programming the STM32 for Time-of-Flight (ToF) sensor operation was the successful initialization and configuration of the necessary hardware peripherals. This required accurately mapping the physical wiring connections (including I2C and GPIO control pins) to the virtual configurations within the STM32 Cube MX environment, followed by correct I2C address assignment in the application code. It took a good while to debug and manual test while reading documentation for STM32 and the VL53L8 to find the right connections. The VL53L8 ToF sensors are new, and advanced documentation was hard to come by which made the process longer.

3.3.3 Wireless communication

The primary challenge in wireless communication between the web interface and the Raspberry Pi was ensuring successful communication to the ROS 2 whilst also ensuring smooth movement of the robot. This required a change from Flask to ROSLIB due to Flask having an action send rate of 6Hz while ROS 2 had an action expectation rate of 10Hz. This caused the robots movement to jitter and pause. However, ROSLIB ensured proper and better communication with the ROS 2 whilst complying with ROS 2's expectation actions being sent at 10Hz, the mechanism is that ROS 2 publishes feedback at 10Hz, rosbridge_server receives the ROS message and converts it to a JSON message, sends it to the web interface where ROSLIB sends the feedback callback.

4 TESTING AND RESULTS

4.1 Testing

To verify the continuous working of the hardware and software we implement tests along the way.

4.1.1 Circuit and Communication Smoke Check

The primary objective of this test was to verify independent I2C communication for six VL53L8CX sensors connected via a multiplexer. The test setup utilized an STM32 microcontroller to cycle through an 8-channel I2C multiplexer, aiming to validate the hardware wiring and addressing logic.

This test is critical because the VL53L8CX ToF sensors share a default I2C address. Connecting multiple units to a single bus without a multiplexer (MUX) will result in address conflicts. Therefore, it is necessary to validate the STM32 microcontroller can correctly cycle through the 8-channel MUX to isolate and address each sensor individually before proceeding with the full ranging algorithm.

The testing environment utilized an STM32 microcontroller connected to an 8-channel I2C MUX. The firmware was configured to perform a smoke test by initializing all sensors and requesting a single ranging frame from each device.

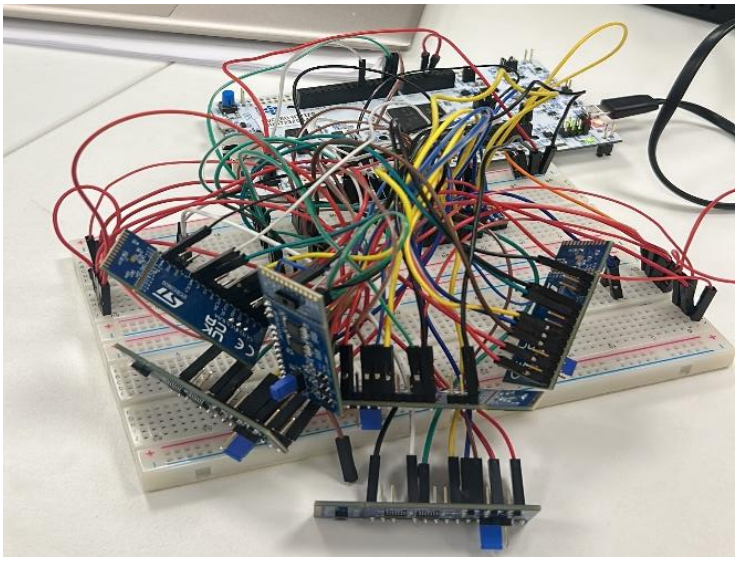


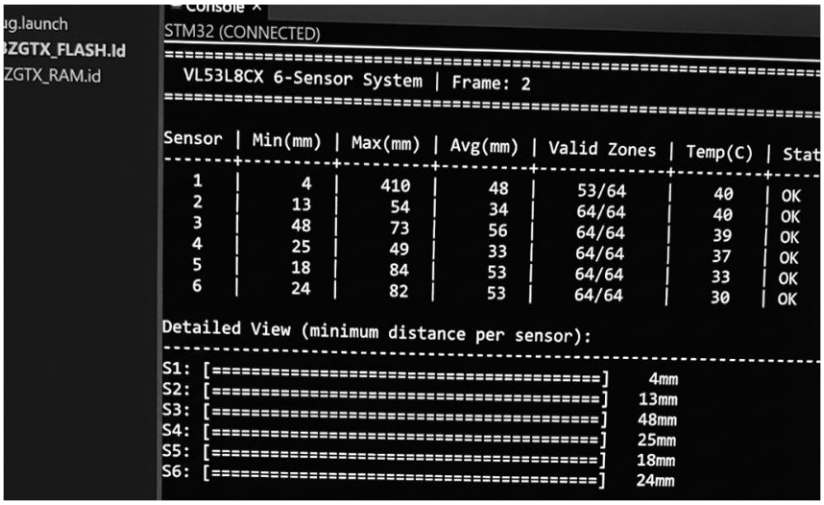
Figure 6. VL53L8CX sensor array connected via I2C multiplexer

4.1.2 Testing the web application

While developing the web application communication inside the local network was frequently tested and confirmed the ability to control the embedded system remotely. These tests were very simple and done in the states of the development where a part of the application logic that was used for communication or control. In the test the device running the application and the iRobot 3 platform were connected to the local network. When connected, user would perform simple movements from the application and with observers would roughly measure the response time and accuracy of the movements.

4.2 Results

4.2.1 Smoke Test Results



```
STM32 (CONNECTED)
==================================================================
  VL53L8CX 6-Sensor System | Frame: 2
==================================================================

Sensor | Min(mm) | Max(mm) | Avg(mm) | Valid Zones | Temp(C) | Stat
-------+---------+---------+---------+-------------+---------+-----
   1   |     4   |   410   |    48   |    53/64    |    40   | OK
   2   |    13   |    54   |    34   |    64/64    |    40   | OK
   3   |    48   |    73   |    56   |    64/64    |    39   | OK
   4   |    25   |    49   |    33   |    64/64    |    37   | OK
   5   |    18   |    84   |    53   |    64/64    |    33   | OK
   6   |    24   |    82   |    53   |    64/64    |    30   | OK

Detailed View (minimum distance per sensor):
------------------------------------------------------------------
S1: [=========================================]    4mm
S2: [=========================================]   13mm
S3: [=========================================]   48mm
S4: [=========================================]   25mm
S5: [=========================================]   18mm
S6: [=========================================]   24mm
```

Figure 7. Six sensor system console log

The console output confirms that the microcontroller successfully addressed all six devices within a single polling cycle. As shown in the system log (Figure 5), distinct data and a status of “OK” were received for each sensor. This shows that the multiplexer correctly isolates channels to prevent addressing errors.

4.2.2 Web application

The team successfully developed a fully functional remote web controller for the robot. This controller is accessible through the main devices IP address and being connected to the Raspberry Pi router, this enables users to operate the robot from multiple devices including computers, phones and tablets. The interface provides essential features for real-time control, including both joystick and keyboard inputs for commanding movement. Furthermore, the application incorporates a connection status monitor to give users immediate feedback on whether the web interface is actively communicating with the ROS 2 system, along with clear error messages for an enhanced user experience. The design prioritizes ease of use, ensuring that no specialized technical knowledge is required for operation (Macenski, T., Foote, B. Gerkey, C. Lalancette, W. Woodall, 2022.).

A core achievement of the web controller is its high performance, which translates to smooth robot movement. This responsiveness is quantified by a measured response time of less than 100 milliseconds from the user’s input (joystick/keyboard) to the robot’s physical movement. This low latency is largely attributable to the system’s architecture, where both the web interface and the robot operate over the Raspberry Pi router, resulting in minimal network latency (Pallets, 2025.) to ensure the fluidity of motion, the system maintains a high velocity update rate of 10 Hz which is crucial for dynamic control. The system has also been tested for reliability, successfully handling multiple connection and disconnection cycles without persistent failures.

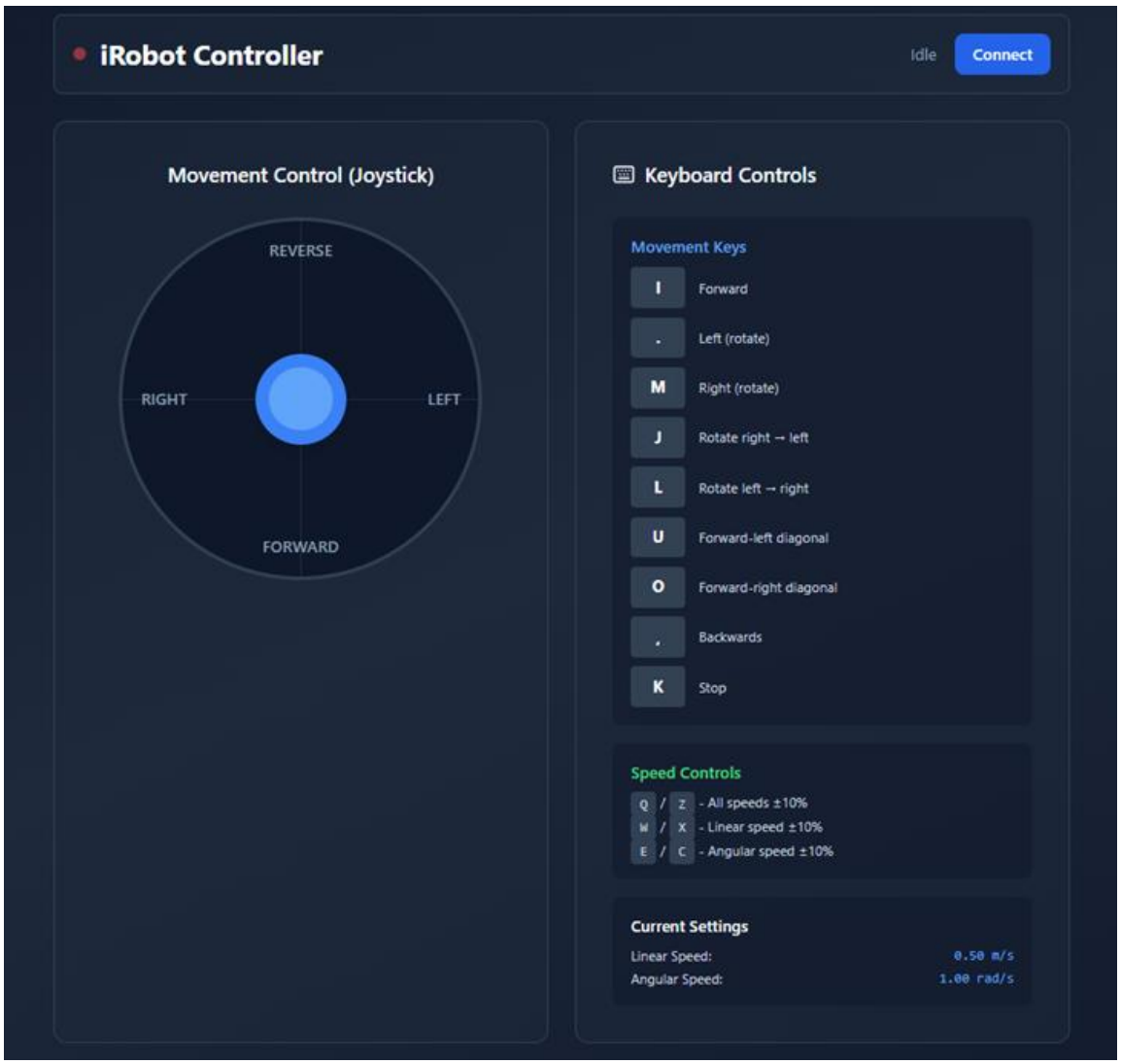


Figure 8. Application

5 DISCUSSION

5.1 Project objectives

The objectives were to produce an embedded IoT system that can be used on the iRobot create 3 platform to turn it into a remote controllable, object avoiding robot using sensors and microcontrollers in a local network controlled wirelessly on a web-interface. This was achieved with great planning and teamwork.

5.2 Contribution of project members

To keep up to date with tasks and contributors at the start of the project a Gantt chart was created.

Initial plan	All	100 %	9.14.25	9.16.25
Research parts	Santeri	100 %	9.14.25	9/14/25
Set up RasPi	Yetunde, Santeri, Nikl	100 %	9.21.25	11.13.25
Ordering parts	Santeri	100 %	9.14.25	9.15.25
Set up STM	Niklas, Santeri, Eemeli	100 %	10.15.25	11.20.25
Set up remote control	Sandesh, Zohaib	100 %	10.2.25	12.4.25
Test sensors with STM	Niklas, Santeri	100 %	10.30.25	11.27.25
Set up sensors on IRobot	Niklas, Santeri	100 %	11.20.25	
Implement sensor fusion	Niklas, Santeri, Eemeli	100 %	11.6.25	11.20.25
Run tests on robot	All	100 %		
Set up application control	Sandesh, Zohaib	100 %	9.25.25	12.5.25
Print Casings	Niklas, Santeri	100 %	11.14.25	11.28.25
Make application	Sandesh, Zohaib	100 %	10.16.25	11.7.25
Design application	Sandesh, Zohaib	100 %	10.2.25	10.30.25
Documentation	Eemeli, Yetunde	100 %	9.11.25	12.8.25
Reporting	All	100 %	10.9.25	12.8.25

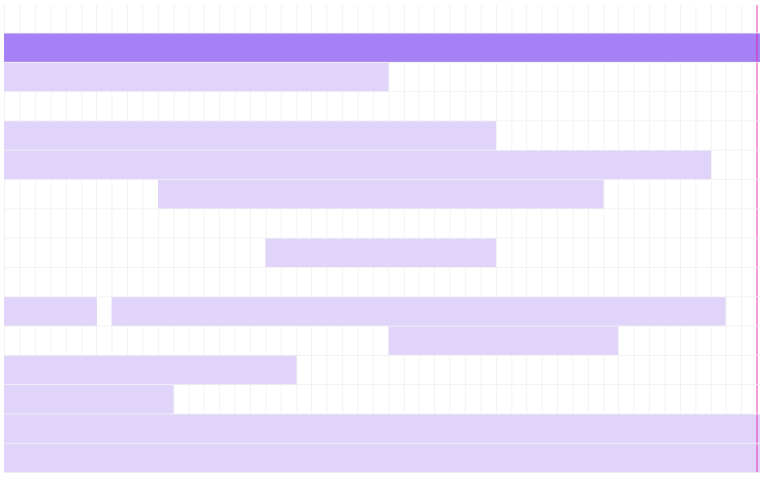


Figure 9. Gantt chart

5.3 Future development

Continuing development of the web interface, a few options are possible to be implemented such as adding a docking and undocking mechanism, a display screen to show where the robot is currently facing and which direction it is heading towards and a metrics table displaying if the sensors currently detect something and force stop the robot to avoid collision.

Collision avoidance / obstacle detection system will be extended to enable simultaneous localization and mapping. The goal is to develop a multifunctional autonomous robot platform.

REFERENCES

Claude 2025. Anthropic. Sonnet 4.5. Programming help, December 2025. https://claude.ai/

Google Gemini 2025. Google. Flash 2.5. Language check, Programming help, December 2025. https://gemini.google.com/.

Flask-CORS Documentation (2025). *Flask-Cors*. Available at: https://flask-cors.readthedocs.io/. Accessed: 17.11.2025.

iRobot Education 2022. Getting Started with the Create 3 Education Robot Guide. https://iroboteducation.github.io/create3_docs/lessons/Create3-Welcome-Guide.pdf. Accessed 2.12.2025

Meta 2025. *React – The library for web and native user interfaces*. https://react.dev/. Accessed: 19.11.2025.

Open Robotics 2025. *ROS 2 Documentation: Kilted documentation*. https://docs.ros.org/en/kilted/index.html. Accessed: 02.12.2025.

Pallets 2025. Flask Documentation (Latest Version). https://flask.palletsprojects.com/. Accessed: 28.10.2025.

Python Software Foundation 2025. Python 3 Documentation. https://docs.python.org/3/. Accessed: 28.10.2025.

Robot Web Tools 2025. roslibjs: The JavaScript library for ROS. Available at: http://wiki.ros.org/roslibjs. Accessed: 03.12.2025.

STMicroelectronics 2023. UM3109: A guide for using the VL53L8CX, low-power, high-performance Time-of-Flight multizone ranging sensor. STMicroelectronics. https://www.st.com/resource/en/user_manual/um3109-a-guide-for-using-the-vl53l8cx-lowpower-highperformance-timeofflight-multizone-ranging-sensor-stmicroelectronics.pdf. Accessed 7.12.2025.

STMicroelectronics 2025. STM32H723ZG MCU Product overview. https://www.st.com/en/evaluation-tools/nucleo-h723zg.html. Accessed 06.12.2025.

Tailwind Labs 2025. Tailwind CSS Documentation. https://tailwindcss.com/docs. Accessed: 19.11.2025.

ZohSal. (2025.). IOT-Project-1 [Source code]. GitHub. https://github.com/ZohSal/IOT-Project-1

APPENDIX 1: CREATIVE COMMONS LICENCES – IMAGE COPYRIGHT